

EXPERIMENT 20

Title: Write a Prolog Program for PLANETS DB

Aim

To develop a Prolog program to create a database containing information about planets and retrieve the required information using Prolog queries.

Objective

- To understand the concept of facts and predicates in Prolog.
- To create a simple database containing information about planets.
- To store properties such as planet name, position, and type.
- To retrieve information from the database using queries.
- To understand logical querying in Prolog.

Theory

A Prolog database can be created using facts that represent information about objects. In this experiment, information about the planets in the Solar System is stored using Prolog predicates.

The predicate `planet/3` is used to represent the planet name, its position from the Sun, and its type.

For example:

`planet(earth, 3, terrestrial).`

This represents that Earth is the third planet from the Sun and is a terrestrial planet.

Prolog allows the stored information to be retrieved by using queries with specific values or variables.

Algorithm

Step 1: Start the program.

Step 2: Define the `planet/3` predicate.

Step 3: Store the names, positions, and types of planets as facts.

Step 4: Define rules to retrieve the required information.

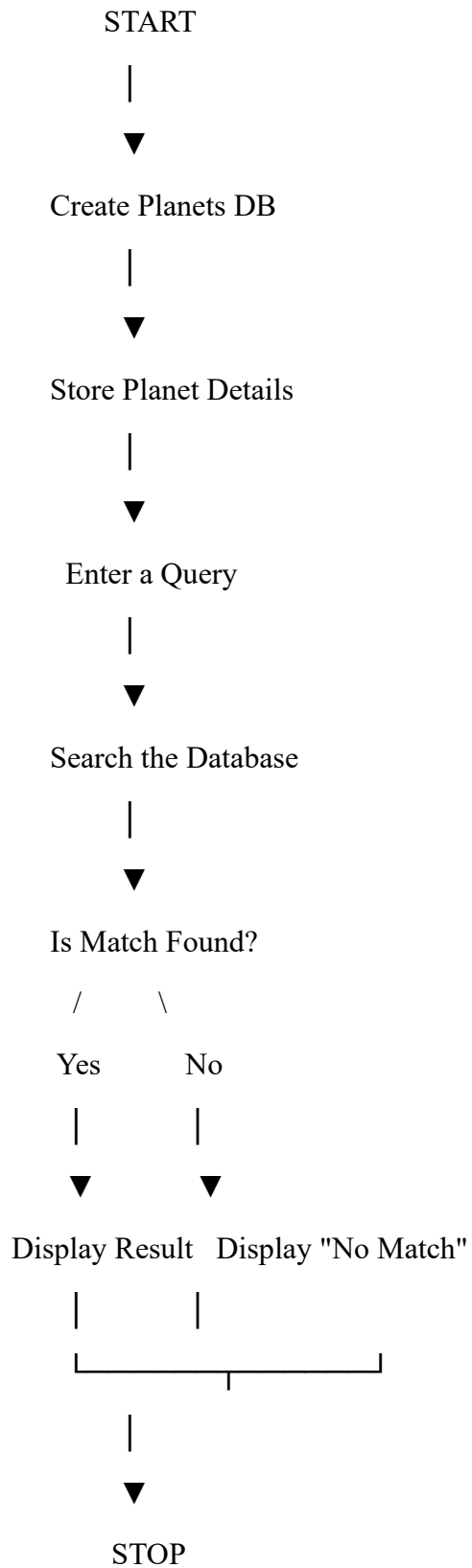
Step 5: Enter a query to search the database.

Step 6: Prolog matches the query with the stored facts.

Step 7: Display the matching result.

Step 8: Stop the program.

Flowchart



Prolog Program

% Planets Database

planet(mercury, 1, terrestrial).

planet(venus, 2, terrestrial).

planet(earth, 3, terrestrial).

planet(mars, 4, terrestrial).

planet(jupiter, 5, gas_giant).

planet(saturn, 6, gas_giant).

planet(uranus, 7, ice_giant).

planet(neptune, 8, ice_giant).

% Rule to find the position of a planet

find_position(Planet, Position) :-

planet(Planet, Position, _).

% Rule to find the type of a planet

find_type(Planet, Type) :-

planet(Planet, _, Type).

Queries

Query 1: Find the position of Earth

?- find_position(earth, Position).

Sample Output

Position = 3.

Query 2: Find the type of Jupiter

?- find_type(jupiter, Type).

Sample Output

Type = gas_giant.

Query 3: Find all terrestrial planets

?- planet(Planet, Position, terrestrial).

Sample Output

Planet = mercury,

Position = 1 ;

Planet = venus,

Position = 2 ;

Planet = earth,

Position = 3 ;

Planet = mars,

Position = 4 ;

false.

Query 4: Display all planets

?- planet(Planet, Position, Type).

Sample Output

Planet = mercury,

Position = 1,

Type = terrestrial ;

Planet = venus,

Position = 2,

Type = terrestrial ;

Planet = earth,

Position = 3,

Type = terrestrial ;

Planet = mars,
Position = 4,
Type = terrestrial ;

Planet = jupiter,
Position = 5,
Type = gas_giant ;

Planet = saturn,
Position = 6,
Type = gas_giant ;

Planet = uranus,
Position = 7,
Type = ice_giant ;

Planet = neptune,
Position = 8,
Type = ice_giant.

Explanation

The predicate planet(Planet, Position, Type) contains three arguments:

- **Planet** – Name of the planet.
- **Position** – Position of the planet from the Sun.
- **Type** – Classification of the planet.

For example:

planet(earth, 3, terrestrial).

means that Earth is the third planet from the Sun and belongs to the terrestrial planet category.

The rule:

find_position(Planet, Position) :-

planet(Planet, Position, _).

is used to find the position of a particular planet.

The rule:

find_type(Planet, Type) :-

planet(Planet, _, Type).

is used to find the type of a particular planet.

The underscore _ represents a value that is not required in the query.

Advantages

- Simple and structured representation of planetary information.
- Easy retrieval using queries.
- Demonstrates the use of Prolog facts and rules.
- Supports logical searching.
- Easy to extend by adding more planetary information.

Applications

- Educational knowledge bases.
- Astronomy information systems.
- Expert systems.
- Knowledge representation.
- Artificial Intelligence applications.
- Database-based reasoning systems.

Result

The Prolog program was successfully implemented to create a database of planets and retrieve information such as their position and type using Prolog queries.

Conclusion

Thus, the PLANETS database was successfully implemented using Prolog. This experiment demonstrates the use of facts, predicates, rules, variables, and queries for representing and retrieving information from a knowledge base.

```

5
6
7 # Display all planets
8 def display_planets():
9     print("\nPLANETS DATABASE")
10    print("-" * 55)
11
12    for planet, details in planets.items():
13        print("Planet:", planet)
14        print("Type:", details["type"])
15        print("Distance from Sun:", details["distance"], "million km")
16        print("Number of Moons:", details["moons"])
17    print("-" * 55)
18
19
20 # Search for a planet
21 def search_planet(name):
22     if name in planets:
23         details = planets[name]
24
25         print("\nPlanet:", name)
26         print("Type:", details["type"])
27         print("Distance from Sun:", details["distance"], "million km")
28         print("Number of Moons:", details["moons"])
29     else:
30         print("Planet not found.")
31
32
33 # Main program
34 while True:
35     print("\n--- PLANETS DATABASE ---")
36     print("1. Display all planets")
37     print("2. Search for a planet")
38     print("3. Exit")
39
40     choice = input("Enter your choice: ")
41
42     if choice == "1":
43         display_planets()
44
45     elif choice == "2":
46         name = input("Enter planet name: ").strip().capitalize()
47         search_planet(name)
48
49     elif choice == "3":
50         print("Program terminated.")
51         break
52
53     else:
54         print("Invalid choice.")

```

--- PLANETS DATABASE ---

1. Display all planets

2. Search for a planet

3. Exit

Enter your choice: 2

Enter planet name: Earth

Planet: Earth

Type: Terrestrial

Distance from Sun: 149.6 million km

Number of Moons: 1

C:\Users\Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.py > ...

```
1  # PLANETS DATABASE
2
3  planets = {
4      "Mercury": {
5          "type": "Terrestrial",
6          "distance": 57.9,
7          "moons": 0
8      },
9      "Venus": {
10         "type": "Terrestrial",
11         "distance": 108.2,
12         "moons": 0
13     },
14     "Earth": {
15         "type": "Terrestrial",
16         "distance": 149.6,
17         "moons": 1
18     },
19     "Mars": {
20         "type": "Terrestrial",
21         "distance": 227.9,
22         "moons": 2
23     },
24     "Jupiter": {
25         "type": "Gas Giant",
26         "distance": 778.5,
27         "moons": 95
28     },
29     "Saturn": {
30         "type": "Gas Giant",
31         "distance": 1434,
32         "moons": 146
33     },
34     "Uranus": {
35         "type": "Ice Giant",
36         "distance": 2871,
37         "moons": 28
38     },
39     "Neptune": {
40         "type": "Ice Giant",
41         "distance": 4495,
42         "moons": 16
43     }
44 }
45
46
```

--- PLANETS DATABASE ---

1. Display all planets
2. Search for a planet
3. Exit

Enter your choice: 2

Enter planet name: Earth

Planet: Earth

Type: Terrestrial

Distance from Sun: 149.6 million km

Number of Moons: 1